Dylan Francis DAAN 682
29MAR2026

Homework_10:

Code:

```
#
========S2==========================================================================
=
# DAAN: 682: Data Analytics Programming in Python
# Author: Dylan Francis
# Title: Homework_10: Unsupervised Learning with SciKit Learn III: Adcanced Models
#
==============================================================================
#
#
==============================================================================
# Perform exploratory data analysis on the Breast Cancer Data and interpret the results. (10
points)
# Build and evaluate SVM (Support Vector Machine) models using different kernel functions.
Report the performance for each kernel and interpret the results. (40 points)
# Build Random Forest models with different values of n_estimators. Report the accuracy for
each value and determine the optimal n_estimators. (25 points)
# Build AdaBoost models with different values of n_estimators. Report the accuracy for each
value and determine the optimal n_estimators. (25 points)
#
==============================================================================

import os
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn import svm
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier

path = r"C:\Users\dylan\OneDrive\Documents\GRAD_SCHOOL\DAAN_682\HOMEWORK_8"
os.chdir(path)
breastcancer = pd.read_csv("breastcancer.csv")
target = "Classification"

#
==============================================================================
#1.) Data exploratory
print(f"\nThere are {breastcancer.shape[0]} rows, and {breastcancer.shape[1]} columns.")
```

```
print("\n=======DATASET_INFO======")
print(breastcancer.info())
print("\n=======SUMMARY_STATS======")
print(breastcancer.describe())
breastcancer_corr = breastcancer.corr()
print(f"\n Here is the correlation martrix: {breastcancer_corr}")
print("\n===== MISSING VALUES =====")
print(f"{breastcancer.isnull().sum()}")
print("\n===== DUPLICATE ROWS =====")
print(f"The total number of duplicate rows is: {breastcancer.duplicated().sum()}")

#
==============================================================================
#2.)
features = breastcancer.drop(columns=[target])
targets = breastcancer[target]
scaler = StandardScaler()

#split the data into 80% training and 20% test
x_train, x_test, y_train, y_test = train_test_split(
    features, targets, test_size=0.2, random_state=13, stratify=targets)

#scale the data so no data "weighs" too heavily
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

#linear model
svm_linear = svm.SVC(kernel = 'linear')
svm_linear.fit(x_train_scaled, y_train)

#rbf model
svm_rbf = svm.SVC(kernel = 'rbf', gamma = 'scale')
svm_rbf.fit(x_train_scaled, y_train)

# Poly model
svm_poly = svm.SVC(kernel = 'poly', degree =3)
svm_poly.fit(x_train_scaled, y_train)

# Predications
predictions = {
    "Linear_SVM": svm_linear.predict(x_test_scaled),
    "RBF_SVM": svm_rbf.predict(x_test_scaled),
    "Poly_SVM": svm_poly.predict(x_test_scaled)
}
```

```
# Results
results = []

for name, preds in predictions.items():
   print(f"\nCLASSIFICATION REPORT: {name}")
   print(classification_report(y_test, preds))

print("""The classification report suggest very similar results for the Linear and RBF models.
     These two models yield roughly the same precision, recall, F-1 scores, and accuracy. The
     model that is distinctly different, and produces worse performance on this data set, is the
poly model,
     which performed worse across the board. """)

#
==============================================================================
#3.) random forest

# I started with a wide range of n_values, [10, 50, 100, 200, 400, 800, 1000].
# I literated on this list until I got in the range of the highest accuracy.
n_values = list(range(1,31))
rf_results = []

for n in n_values:
   rf_model = RandomForestClassifier(n_estimators=n, random_state = 42)
   rf_model.fit(x_train, y_train)
   preds = rf_model.predict(x_test)

   accuracy = metrics.accuracy_score(y_test, preds)
   rf_results.append({
      "n_estimators": n,
      "Accuracy": accuracy
      })

rf_results_df = pd.DataFrame(rf_results)
print("\n===== Random Forest Summary =====")
print(rf_results_df)
print("""I iterated on the number of trees several time to determine the optimal number of
estimators.
     Several numbers produced the highest accurary of 83%. The number of Ns that yeilded this
result was 13-26,
     with an exception of 23, which had a small drop.""")
```

```
#
==============================================================================
#4.) AdaBoost models
n_values_boots = list(range(1,51))
boost_results = []

for n in n_values_boots:
    boost_model = AdaBoostClassifier(n_estimators=n, random_state=42)
    boost_model.fit(x_train_scaled, y_train)
    boost_model.predict(x_test_scaled)
    preds_boost = boost_model.predict(x_test_scaled)

    boost_acc = metrics.accuracy_score(y_test, preds_boost)
    boost_results.append({
        "n_estimators":n,
        "Accuracy": boost_acc
        })
boost_results_df = pd.DataFrame(boost_results)
print("\n===== AdaBoost Summary =====")
print(boost_results_df)
print("""I get the highest accuracy when n = 16, 18, and 19. The model appears to drop in
accuracy after that,
      which is largely expected due to the weight given the misclassifcation errors.""")
```

Terminal:

```
C:/Users/dylan/OneDrive/Documents/GRAD_SCHOOL/DAAN_682/HOMEWORK_10/homework
10.py --wdir

There are 116 rows, and 10 columns.

=======DATASET_INFO======
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 116 entries, 0 to 115
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Age           116 non-null    int64
 1   BMI           116 non-null    float64
 2   Glucose       116 non-null    int64
 3   Insulin       116 non-null    float64
 4   HOMA          116 non-null    float64
 5   Leptin        116 non-null    float64
 6   Adiponectin   116 non-null    float64
 7   Resistin      116 non-null    float64
```

```
 8   MCP.1          116 non-null    float64
 9   Classification  116 non-null    int64
dtypes: float64(7), int64(3)
memory usage: 9.2 KB
None

=======SUMMARY_STATS======
            Age        BMI  ...       MCP.1  Classification
count  116.000000  116.000000  ...   116.000000      116.000000
mean    57.301724   27.582111  ...   534.647000        1.551724
std     16.112766    5.020136  ...   345.912663        0.499475
min     24.000000   18.370000  ...    45.843000        1.000000
25%     45.000000   22.973205  ...   269.978250        1.000000
50%     56.000000   27.662416  ...   471.322500        2.000000
75%     71.000000   31.241442  ...   700.085000        2.000000
max     89.000000   38.578759  ...  1698.440000        2.000000

[8 rows x 10 columns]

 Here is the correlation martrix:                    Age       BMI  ...     MCP.1  Classification
Age            1.000000  0.008530  ...  0.013462       -0.043555
BMI            0.008530  1.000000  ...  0.224038       -0.132586
Glucose        0.230106  0.138845  ...  0.264879        0.384315
Insulin        0.032495  0.145295  ...  0.174356        0.276804
HOMA           0.127033  0.114480  ...  0.259529        0.284012
Leptin         0.102626  0.569593  ...  0.014009       -0.001078
Adiponectin   -0.219813 -0.302735  ... -0.200694       -0.019490
Resistin       0.002742  0.195350  ...  0.366474        0.227310
MCP.1          0.013462  0.224038  ...  1.000000        0.091381
Classification -0.043555 -0.132586  ...  0.091381        1.000000

[10 rows x 10 columns]

===== MISSING VALUES =====
Age             0
BMI             0
Glucose         0
Insulin         0
HOMA            0
Leptin          0
Adiponectin     0
Resistin        0
MCP.1           0
Classification  0
```

dtype: int64

===== DUPLICATE ROWS =====
The total number of duplicate rows is: 0

CLASSIFICATION REPORT: Linear_SVM

	precision	recall	f1-score	support
1	0.77	0.91	0.83	11
2	0.91	0.77	0.83	13
accuracy			0.83	24
macro avg	0.84	0.84	0.83	24
weighted avg	0.84	0.83	0.83	24

CLASSIFICATION REPORT: RBF_SVM

	precision	recall	f1-score	support
1	0.82	0.82	0.82	11
2	0.85	0.85	0.85	13
accuracy			0.83	24
macro avg	0.83	0.83	0.83	24
weighted avg	0.83	0.83	0.83	24

CLASSIFICATION REPORT: Poly_SVM

	precision	recall	f1-score	support
1	0.75	0.27	0.40	11
2	0.60	0.92	0.73	13
accuracy			0.62	24
macro avg	0.68	0.60	0.56	24
weighted avg	0.67	0.62	0.58	24

The classification report suggest very similar results for the Linear and RBF models.
These two models yield roughly the same precision, recall, F-1 scores, and accuracy. The model that is distinctly different, and produces worse performance on this data set, is the poly model,
which performed worse across the board.

===== Random Forest Summary =====

	n_estimators	Accuracy
0	1	0.541667
1	2	0.541667
2	3	0.625000
3	4	0.708333
4	5	0.750000
5	6	0.750000
6	7	0.791667
7	8	0.708333
8	9	0.750000
9	10	0.750000
10	11	0.750000
11	12	0.750000
12	13	0.833333
13	14	0.833333
14	15	0.833333
15	16	0.833333
16	17	0.833333
17	18	0.833333
18	19	0.833333
19	20	0.833333
20	21	0.833333
21	22	0.833333
22	23	0.791667
23	24	0.833333
24	25	0.833333
25	26	0.833333
26	27	0.750000
27	28	0.750000
28	29	0.750000
29	30	0.750000

I iterated on the number of trees several time to determine the optimal number of estimators.

Several numbers produced the highest accurary of 83%. The number of Ns that yeilded this result was 13-26,

with an exception of 23, which had a small drop.

===== AdaBoost Summary =====

	n_estimators	Accuracy
0	1	0.750000
1	2	0.583333
2	3	0.708333
3	4	0.708333
4	5	0.708333
5	6	0.708333

6	7	0.708333
7	8	0.708333
8	9	0.708333
9	10	0.708333
10	11	0.708333
11	12	0.708333
12	13	0.708333
13	14	0.708333
14	15	0.708333
15	16	0.750000
16	17	0.708333
17	18	0.750000
18	19	0.750000
19	20	0.708333
20	21	0.666667
21	22	0.708333
22	23	0.666667
23	24	0.666667
24	25	0.666667
25	26	0.708333
26	27	0.666667
27	28	0.666667
28	29	0.666667
29	30	0.666667
30	31	0.666667
31	32	0.666667
32	33	0.666667
33	34	0.666667
34	35	0.666667
35	36	0.708333
36	37	0.666667
37	38	0.666667
38	39	0.666667
39	40	0.666667
40	41	0.666667
41	42	0.666667
42	43	0.666667
43	44	0.666667
44	45	0.666667
45	46	0.666667
46	47	0.666667
47	48	0.666667
48	49	0.666667
49	50	0.708333

Dylan Francis

I get the highest accuracy when n = 16, 18, and 19. The model appears to drop in accuracy after that,

which is largely expected due to the weight given the misclassifcation errors.